

```

# 1. استيراد المكتبات الأساسية لمعالجة وتحليل البيانات والرسوم
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 2. أدوات تقسيم البيانات والتحجيم والترميز
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 3. ومقاييس التقييم Regression مكتبات نماذج ال
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 4. ومقاييس التقييم Classification مكتبات نماذج التصنيف
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# 5. وتقليل الأبعاد Clustering مكتبات التعلم غير الخاضع للإشراف والتجميع
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# ضبط مظهر الرسوم البيانية لتكون واضحة وأنيقة
sns.set_theme(style="whitegrid")
print("✅ تم استيراد جميع المكتبات بنجاح")

```

✅ تم استيراد جميع المكتبات بنجاح!

```

print("--- 1. البدء بمرحلة معالجة البيانات ---")

# 1. تحميل ملف البيانات (جدولة) \t
df = pd.read_csv('marketing_campaign.csv', sep='\t')
print(f"أبعاد البيانات الأصلية: {df.shape}")

# 2. باستخدام القيمة الوسطية Income معالجة القيم المفقودة في الدخل
df['Income'] = df['Income'].fillna(df['Income'].median())

# 3. هندسة الميزات (Feature Engineering)
# حساب العمر بناءً على السنة الحالية للمشروع
df['Age'] = 2026 - df['Year_Birth']

# حساب إجمالي الإنفاق بجمع كافة أعمدة المنتجات المحددة
spending_cols = ['MntWines', 'MntFruits', 'MntMeatProd']
df['TotalSpending'] = df[spending_cols].sum(axis=1)

# حساب إجمالي الأطفال بجمع الأطفال والمراهقين
df['TotalChildren'] = df['Kidhome'] + df['Teenhome']

# 4. تصفية وحذف القيم غير المنطقية (Filtering)

```

```

df = df[(df['Age'] >= 18) & (df['Age'] <= 100)]
df = df[df['Income'] > 0]
print(f"أبعاد البيانات بعد التنظيف والتصفية: {df.shape}")

# 5. ترميز البيانات النصية والفئوية (Encoding)
# أ) Label Encoding لعمود التعليم Education
edu_mapping = {"Basic": 0, "Graduation": 1, "Master": 2}
df['Education'] = df['Education'].map(edu_mapping).fillna(0)

# ب) One-Hot Encoding للحالة الاجتماعية Marital_Status (الباقي)
top_4_marital = df['Marital_Status'].value_counts().in
df['Marital_Status'] = df['Marital_Status'].apply(lambda x:
df = pd.get_dummies(df, columns=['Marital_Status'], drop_

print("✅ تمت مرحلة معالجة وتجهيز البيانات بنجاح")

```

--- البدء بمرحلة معالجة البيانات 1. ---
 أبعاد البيانات الأصلية: (29, 2240)
 أبعاد البيانات بعد التنظيف والتصفية: (32, 2237)
 ✅ تمت مرحلة معالجة وتجهيز البيانات بنجاح!

```

print("--- 2. مرحلة الـ Regression Task ---")

# تحديد الميزات الأربعة والمستهدف المستمر (TotalSpending)
features_reg = ['Age', 'Income', 'Education', 'TotalChildren']
X_reg = df[features_reg]
y_reg = df['TotalSpending']

# تقسيم البيانات بنسبة 80% تدريب و 20% اختبار
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_

# تطبيق تحجيم الميزات المستقلة باستخدام StandardScaler
scaler_reg = StandardScaler()
X_train_reg_scaled = scaler_reg.fit_transform(X_train_reg)
X_test_reg_scaled = scaler_reg.transform(X_test_reg)

# بناء وتشغيل النماذج الثلاثة المطلوبة
reg_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression (alpha=1.0)": Ridge(alpha=1.0),
    "Decision Tree Regressor (max_depth=5)": DecisionTreeRegr
}

reg_results = {}
for name, model in reg_models.items():
    model.fit(X_train_reg_scaled, y_train_reg)
    preds = model.predict(X_test_reg_scaled)

    # حساب مقاييس الأداء المحددة في التكلفة
    mse = mean_squared_error(y_test_reg, preds)
    rmse = np.sqrt(mse)

    r2 = r2_score(y_test_reg, preds)

```

```
reg_results[name] = {"MSE": round(mse, 4), "RMSE": round(rmse, 4), "R2": round(r2, 4)}
```

جدول المقارنة النهائي للأداء #

```
df_reg_results = pd.DataFrame(reg_results).T
print("\n📊 Regression المقارنة أداء نماذج ال")
display(df_reg_results)
```

--- 2. Regression Task مرحلة ال ---

📊 Regression المقارنة أداء نماذج ال:

	MSE	RMSE	R2 Score	
Decision Tree Regressor (max_depth=5)	81435.3332	285.3688	0.7856	✎

```
print("--- 3. Classification Task مرحلة ال ---")
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# أ (Class Imbalance) فحص توزيع الفئات للتحقق من عدم التوازن
print("في البيانات (Response) توزيع فئات الاستجابة:")
print(df['Response'].value_counts())

# ب (TotalSpending ≤ Features) تحديد الميزات لنموذج التصنيف
features_clf = features_reg + ['TotalSpending']
X_clf = df[features_clf]
y_clf = df['Response']

# ج (80/20) تقسيم البيانات بنسبة
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(X_clf, y_clf,
                                                                      test_size=0.2,
                                                                      random_state=42)

# د (Logistic Regression و KNN) المطلوب للـ التحجيم
scaler_clf = StandardScaler()
num_cols_clf = ['Income', 'Age', 'TotalChildren', 'TotalSpending']

X_train_clf_scaled = X_train_clf.copy()
X_test_clf_scaled = X_test_clf.copy()
X_train_clf_scaled[num_cols_clf] = scaler_clf.fit_transform(X_train_clf_scaled[num_cols_clf])
X_test_clf_scaled[num_cols_clf] = scaler_clf.transform(X_test_clf_scaled[num_cols_clf])

# هـ تعريف وتدريب النماذج المطلوبة
models_clf = {
    "Logistic Regression": LogisticRegression(class_weight='balanced'),
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5),
    "Random Forest Classifier": RandomForestClassifier(n_estimators=100)
}

# و تقييم النماذج وطباعة التقارير ومصفوفة الارتباك
for name, model in models_clf.items():
    print(f"\n===== {name} =====")
```

```

# لذا نمرر لها البيانات الأصلية بحسب أتمتة ال
if name == "Random Forest Classifier":
    model.fit(X_train_clf, y_train_clf)
    preds = model.predict(X_test_clf)
else:
    model.fit(X_train_clf_scaled, y_train_clf)
    preds = model.predict(X_test_clf_scaled)
print("تقرير التصنيف (Classification Report):")
print(classification_report(y_test_clf, preds))
print("مصفوفة الارتباك (Confusion Matrix):")
print(confusion_matrix(y_test_clf, preds))

```

--- 3. Classification Task ---

في البيانات (Response) توزيع فئات الاستجابة:

Response

0 1903

1 334

Name: count, dtype: int64

===== Logistic Regression =====

===== K-Nearest Neighbors =====

===== Random Forest Classifier =====

تقرير التصنيف (Classification Report):

	precision	recall	f1-score	support
0	0.87	0.95	0.91	376
1	0.51	0.25	0.34	72
accuracy			0.84	448
macro avg	0.69	0.60	0.62	448
weighted avg	0.81	0.84	0.82	448

مصفوفة الارتباك (Confusion Matrix):

```

[[359  17]
 [ 54  18]]

```

print("--- 4. Clustering Task ---")

from sklearn.cluster import KMeans

from sklearn.decomposition import PCA

اختيار الميزات الأربعة المطلوبة للتجميع وتحجيمها (أ)

```

cluster_features = ['TotalSpending', 'Income', 'Age',
X_cluster = df[cluster_features].copy()

```

scaler_cluster = StandardScaler()

X_cluster_scaled = scaler_cluster.fit_transform(X_cluster)

المثالية k لتحديد قيمة (Elbow Method) (ب) استخدام طريقة الكوع

inertia = []

K_range = range(1, 11)

for k in K_range:

kmeans = KMeans(n_clusters=k, random_state=42, n_init=1000)

kmeans.fit(X_cluster_scaled)

inertia.append(kmeans.inertia_)

```

inertia.append(kmeans.inertia_)

# رسم منحني الكوع بيانياً
plt.figure(figsize=(8, 4))
plt.plot(K_range, inertia, 'bo-')
plt.xlabel('عدد المجموعات (k)')
plt.ylabel('Inertia / WCSS')
plt.title('عدد العدد الأمثل للمجموعات (Elbow Method) طريقة الكوع')
plt.grid(True)
plt.show()

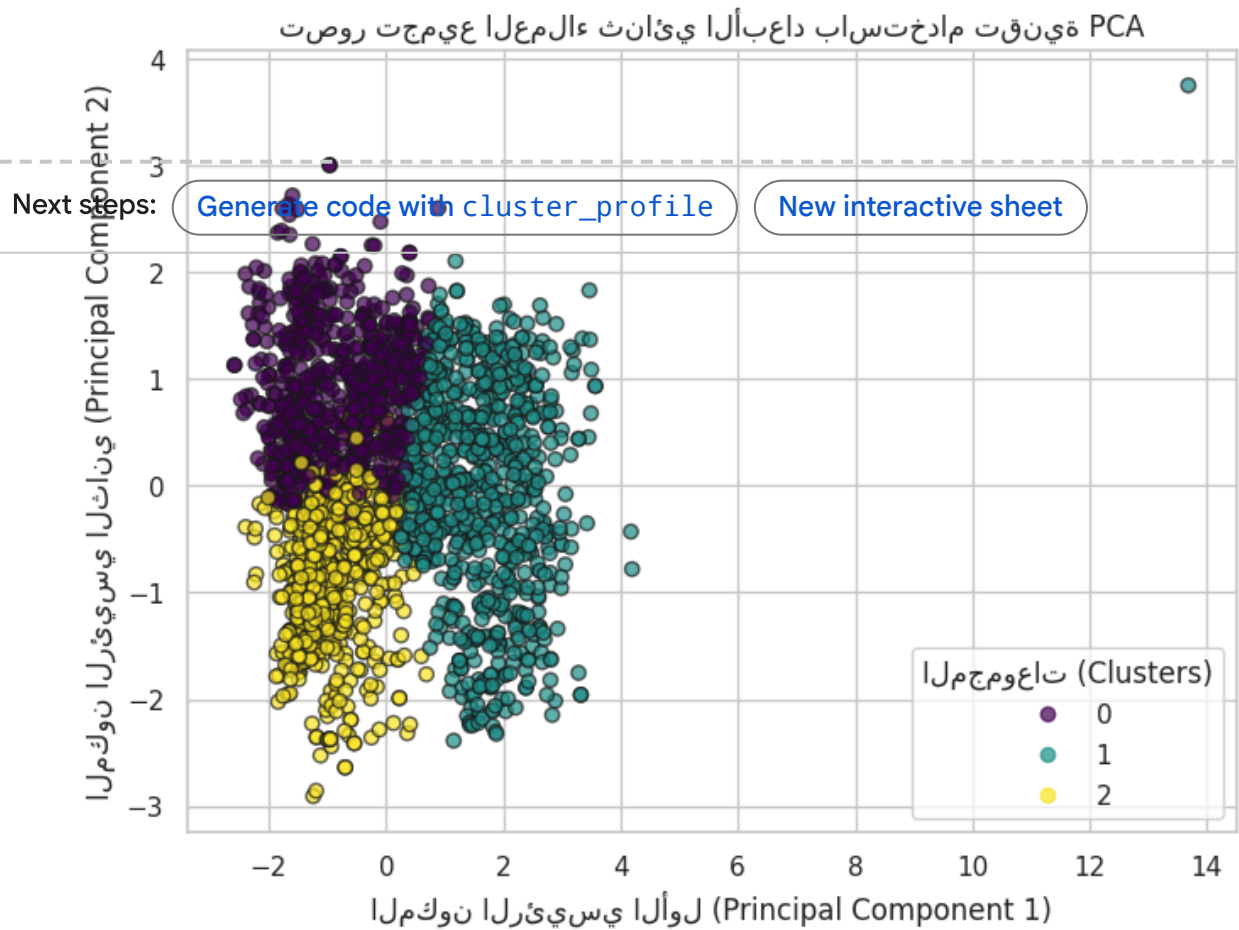
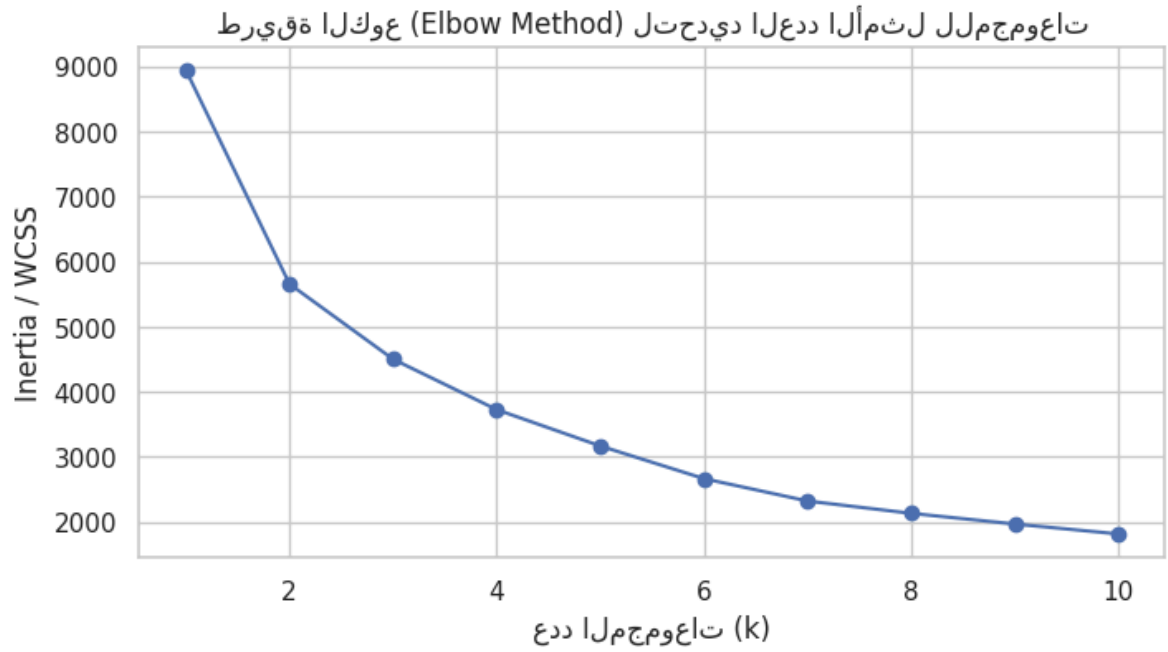
# (أ) بناءً على العدد المختار K-Means (ج) تطبيق خوارزمية
optimal_k = 3
kmeans_final = KMeans(n_clusters=optimal_k, random_state=42)
df['Cluster'] = kmeans_final.fit_predict(X_cluster_scaled)

# تقليل الأبعاد ورسم المجموعات بيانياً ثنائي الأبعاد باستخدام
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_cluster_scaled)
plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df['Cluster'])
plt.legend(*scatter.legend_elements(), title="المجموعات")
plt.title('PCA تصور تجميع العملاء ثنائي الأبعاد باستخدام تقنية')
plt.xlabel('المكون الرئيسي الأول (Principal Component 1)')
plt.ylabel('المكون الرئيسي الثاني (Principal Component 2)')
plt.grid(True)
plt.show()

# (هـ) تحليل وفهم خصائص كل مجموعة (Cluster Profiling)
cluster_profile = df.groupby('Cluster')[cluster_features].mean()
print("\n--- متوسط ميزات العملاء لكل مجموعة تفصيلية ---")
display(cluster_profile)

```

--- 4. مرحلة ال Clustering Task ---



--- متوسط مييزات العملاء لكل مجموعة تفصيلية ---